Individual Report — Reinforcement Learning Project

DQN on `highway-v0`: Implementation, Infrastructure, and Analysis

Maxence Rossignol
Group 17 — CentraleSupélec, Mention IA

April 2026

1 Personal Contribution

My main contributions to the project focused on the **training infrastructure**, **experimental pipeline**, and **cluster deployment**. Specifically:

- **Fair comparison framework**: I converted the DQN training loop from an episode-based to a **timestep-based** interface, so that both our DQN and the Stable-Baselines3 (SB3) DQN are trained for the same number of environment steps. This ensures the training curves share the same x-axis and the comparison is transparent.
- **Incremental result saving**: I implemented CSV-based logging that saves training rewards and evaluation results after each seed. This allows partial runs to still produce usable data—a design choice that proved critical when our first cluster job was killed mid-training.
- **DCE cluster deployment**: I set up the full training pipeline on CentraleSupélec's distributed computing cluster (SLURM-based), including conda environment creation with CUDA 12.4 support, batch job scripts with configurable parameters, and monitoring.
- **Debugging and fixes**: I identified and fixed several issues in the pipeline, including a missing `Monitor` wrapper for SB3 reward logging, an observation space mismatch caused by `configure()` requiring a `reset()` before `DummyVecEnv` construction, and checkpoint saving during training rather than only at the end.

2 Experiments

All experiments use the instructor-provided configuration: `highway-v0` with Kinematics observations (10×5 matrix), `DiscreteMetaAction` (5 actions), 45 vehicles, 4 lanes, 30-second episodes, and normalized rewards.

Both agents share the same core hyperparameters for a fair comparison:

- Learning rate: 10^{-3}, discount factor $\gamma = 0.99$, batch size 64
- Replay buffer: 50,000 transitions, target network update every 500 steps
- Epsilon: linear decay from 1.0 to 0.05 over 50% of training (DQN) / 30% (SB3)
- Network: MLP $50 \to 256 \to 256 \to 5$ for both agents (we set SB3's `policy_kwargs=dict(net_arch=[256, 256])` to match our architecture exactly)

An earlier run (v2) used a smaller default `MlpPolicy` (64 → 64) for SB3 while our DQN used 256 → 128, making the comparison unfair. In the final run (v3), we matched both networks to 256 → 256 and used seeds 52, 53, 54.

I ran experiments on the DCE cluster (NVIDIA GTX 1080 Ti). A first attempt with 3 seeds was killed by SLURM after exceeding the 5-hour time limit. The incremental CSV saving ensured no data was lost for the completed seed. This led us to switch to the `gpu_prod_long` partition (48h limit), where the full run—40,000 timesteps × 3 seeds (52, 53, 54) × 2 agents—completed successfully.

3 Results and Analysis

3.1 Training Curves

Figure 1 shows the training curves averaged across 3 seeds, with the shaded area representing ±1 standard deviation.

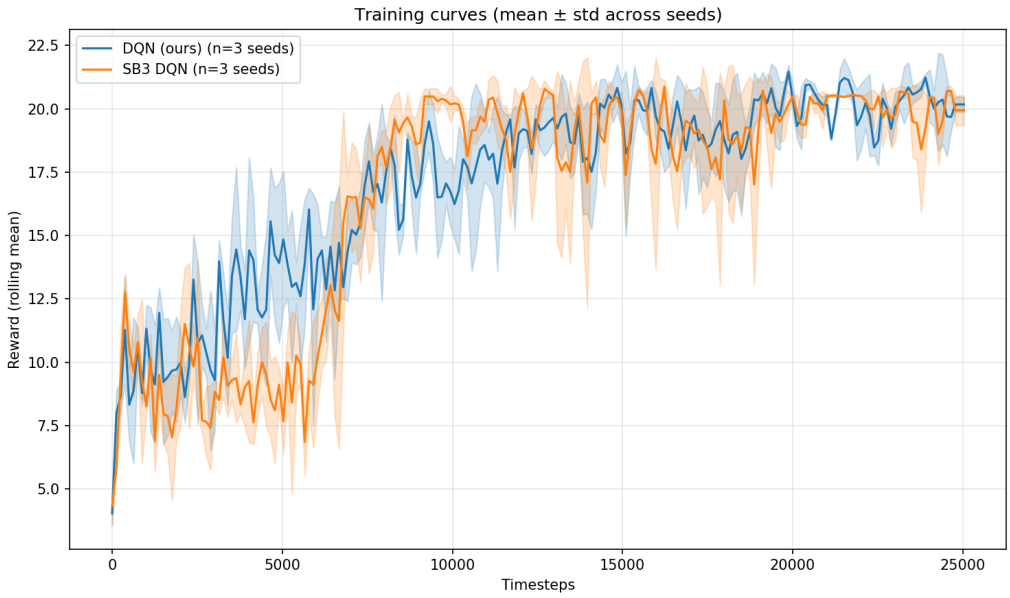

Figure 1: Training curves (mean ± std across seeds 52, 53, 54), 40,000 timesteps. Both agents use identical 256 × 256 MLP architectures.

Key observations:

- Our DQN reaches high reward levels quickly within the first ∼5,000 timesteps but continues to exhibit noisy, high-variance training throughout, with frequent reward drops caused by crash episodes.
- SB3's learning is delayed by its `learning_starts=1000` parameter, but once it begins learning, it converges to a very **stable and consistent** policy with near-zero variance across seeds.
- Both agents converge toward similar mean performance (∼20) by the end of training, but SB3's final policy is significantly more stable.

3.2 Evaluation

Table 1 summarizes the evaluation results over 50 episodes per seed with deterministic policies (no exploration).

Table 1: Evaluation results (50 episodes per seed, deterministic policy).

Agent	Seed	Mean Reward	Std	Crashes	Crash Rate
DQN (ours)	52	19.81	4.52	9/50	18%
DQN (ours)	53	18.99	5.51	12/50	24%
DQN (ours)	54	20.76	3.44	8/50	16%
DQN (ours)	**all**	**19.85**	**4.63**	**29/150**	**19%**
SB3 DQN	52	20.45	0.00	0/50	0%
SB3 DQN	53	20.55	0.05	0/50	0%
SB3 DQN	54	20.45	0.00	0/50	0%
SB3 DQN	**all**	**20.48**	**0.05**	**0/150**	**0%**

With matched architectures, SB3 now outperforms our DQN on stability metrics: higher mean reward (20.48 vs 19.85), dramatically lower variance (0.05 vs 4.63), and **zero crashes** across all 150 evaluation episodes. SB3 learned a very conservative, safe policy that consistently achieves moderate reward without ever colliding. Our DQN achieves higher peak rewards (up to 24.85) but at the cost of a 19% crash rate and high variance across seeds. This reverses our earlier results (v2), where SB3 used a smaller 64×64 network and performed significantly worse—highlighting how **architecture parity is essential for fair comparison**.

3.3 Failure Analysis

Our DQN's 29 crash episodes follow two patterns:

- **Early crashes** (steps 4–8, e.g., reward 2.16 at step 4): the agent attempts a lane change immediately after reset into an occupied lane. These represent catastrophic misjudgments of the initial traffic configuration and produce very low rewards.
- **Late crashes** (steps 19–30, e.g., reward 20.97 at step 30): the agent survives most of the episode but collides near the end, typically during a lane change in dense traffic. These episodes still accumulate a near-full reward before the crash.

SB3's policy, by contrast, exhibits **zero crashes**. Its near-constant reward of $\sim$20.45 suggests it learned a highly conservative strategy—likely maintaining a steady speed and avoiding lane changes entirely. While safe, this policy never reaches the higher rewards (22–25) that our DQN occasionally achieves through more aggressive driving.

4 What Worked and What Failed

What worked:

- The timestep-based unification made the comparison genuinely fair and the training curves directly readable on the same x-axis.
- Incremental CSV saving paid off immediately when our first cluster job timed out. Without it, 2+ hours of compute would have been lost entirely.
- Matching network architectures (v2 $\rightarrow$ v3) was critical: SB3's default 64×64 `MlpPolicy` gave misleadingly poor results. Once we set both agents to 256×256, SB3 demonstrated its strength—its carefully tuned training loop produces more stable policies than our from-scratch implementation.

What failed:

- **Unfair architecture comparison (v2):** our initial results showed our DQN "outperforming" SB3, but SB3 was using a network 4× smaller. This is a cautionary tale about controlling all variables in ML experiments.
- **SB3 reward logging**: the `RewardLoggerCallback` silently produced empty lists because the environment was not wrapped in a `Monitor`. The fix was a single line (`Monitor(env)`).
- **Compute budget estimation**: I underestimated how long 40,000 timesteps × 3 seeds would take on the cluster (∼14 hours), leading to the first job being killed at the SLURM time limit.
- **Observation space bug**: SB3's `DummyVecEnv` reads the observation space at construction time, before `configure()` takes effect. The fix was to call `env.reset()` after `configure()` so the observation space updates from $(5, 5)$ to $(10, 5)$.

5 Limitations

- **No hyperparameter tuning** was performed. Our DQN's 19% crash rate could likely be reduced by tuning the exploration schedule or adding safety-oriented reward shaping. SB3 benefits from its well-tuned training loop (gradient clipping, learning rate scheduling, etc.) which our implementation lacks.
- **SB3's conservative policy**: SB3 achieves zero crashes but with near-constant reward (∼20.45), suggesting it avoids lane changes entirely. A more nuanced analysis would require logging per-step actions to confirm this hypothesis.
- The **extension task** has not yet been completed. Future work could explore Double-DQN and Prioritized Experience Replay as natural improvements to the current DQN implementation.
- The failure analysis is qualitative. A more rigorous approach would involve recording the full observation trajectory leading to each crash to identify systematic failure patterns.